LAPORAN

WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)

SEMESTER III



BKPM Acara 17 API CRUD

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi web mendorong penggunaan aplikasi yang mampu melakukan pertukaran data secara terstruktur antara client dan server. Salah satu teknologi yang digunakan untuk mendukung proses tersebut adalah Application Programming Interface (API). API memungkinkan suatu aplikasi mengakses dan mengelola data melalui mekanisme komunikasi yang telah ditentukan.

RESTful API merupakan salah satu pendekatan dalam pengembangan API yang memanfaatkan protokol HTTP untuk menjalankan operasi terhadap sumber daya. Metode HTTP seperti GET, POST, PUT, dan DELETE digunakan untuk mengambil, menambahkan, memperbarui, dan menghapus data. Data yang dikirimkan oleh API dapat menggunakan format JSON sehingga mudah diproses oleh berbagai aplikasi.

Pada praktikum Acara 17, dilakukan pengembangan RESTful API pada project Sistem Informasi Akademik menggunakan PHP dan MySQL. Implementasi mencakup pengelolaan data mahasiswa, pemrosesan permintaan HTTP, pengolahan data melalui model dan controller, serta pengujian endpoint menggunakan Postman. Praktikum ini juga memperkuat pemahaman mengenai pola arsitektur Model-View-Controller (MVC), routing, dan validasi data.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam praktikum ini adalah sebagai berikut.

1. Bagaimana mengimplementasikan RESTful API menggunakan PHP dan MySQL?
2. Bagaimana menerapkan metode HTTP GET, POST, PUT, dan DELETE untuk pengelolaan data mahasiswa?
3. Bagaimana mengatur alur permintaan API melalui router, controller, dan model?
4. Bagaimana melakukan pengujian endpoint API menggunakan Postman?
5. Bagaimana menerapkan validasi data dan mengembangkan resource baru pada API?

1.3 Tujuan Praktikum

Tujuan praktikum Acara 17 adalah memahami dan mengimplementasikan RESTful API menggunakan PHP dan MySQL, menerapkan operasi CRUD melalui metode HTTP, memahami alur pemrosesan permintaan melalui arsitektur MVC, serta menguji API menggunakan Postman. Selain itu, praktikum ini bertujuan meningkatkan kemampuan dalam melakukan validasi data dan mengembangkan endpoint sesuai kebutuhan aplikasi.

1.4 Manfaat Praktikum

Praktikum ini memberikan pengalaman dalam membangun layanan API yang dapat digunakan untuk mengelola data secara terstruktur. Melalui praktikum ini, mahasiswa dapat memahami hubungan antara client, server, dan basis data, serta mengetahui cara menguji permintaan HTTP dan memeriksa respons yang diberikan oleh API.

BAB 2. LANDASAN TEORI

2.1 Application Programming Interface (API)

Application Programming Interface (API) merupakan antarmuka yang memungkinkan dua perangkat lunak berkomunikasi dan bertukar data. Dalam aplikasi web, API berperan sebagai perantara antara client dan server sehingga client dapat meminta atau mengirimkan data melalui aturan komunikasi tertentu.

API memungkinkan fungsi suatu aplikasi digunakan oleh aplikasi lain tanpa mengharuskan client mengakses basis data secara langsung. Dengan demikian, proses komunikasi dan pengelolaan data dapat dilakukan secara lebih terstruktur.

2.2 RESTful API

RESTful API merupakan pendekatan pengembangan API yang menerapkan prinsip Representational State Transfer (REST). Pendekatan ini menggunakan sumber daya yang dapat diakses melalui endpoint dan memanfaatkan metode HTTP untuk menjalankan operasi tertentu.

RESTful API umumnya menggunakan URL untuk menunjukkan sumber daya yang diakses dan JSON sebagai format pertukaran data. Penerapan RESTful API membantu memisahkan proses pengelolaan data pada server dari aplikasi yang menggunakan data tersebut.

2.3 Metode HTTP

HTTP menyediakan beberapa metode yang digunakan untuk menentukan tindakan terhadap sumber daya pada server. Dalam RESTful API, metode yang umum digunakan meliputi:

1. GET, digunakan untuk mengambil atau membaca data dari server.
2. POST, digunakan untuk mengirimkan data baru agar dapat ditambahkan ke server.
3. PUT, digunakan untuk memperbarui sumber daya yang sudah tersedia.
4. DELETE, digunakan untuk menghapus sumber daya berdasarkan identitas tertentu.

Penggunaan metode HTTP yang sesuai membantu membuat fungsi endpoint lebih jelas dan memudahkan proses pengujian API.

2.4 JavaScript Object Notation (JSON)

JavaScript Object Notation (JSON) merupakan format pertukaran data yang menggunakan struktur pasangan key-value. JSON banyak digunakan dalam komunikasi API karena bentuknya sederhana, mudah dibaca, dan didukung oleh berbagai bahasa pemrograman.

Dalam PHP, fungsi `json_encode()` digunakan untuk mengubah data menjadi format JSON, sedangkan `json_decode()` digunakan untuk membaca atau menguraikan data JSON yang diterima dari client.

2.5 PHP

PHP merupakan bahasa pemrograman sisi server yang digunakan untuk mengembangkan aplikasi web dinamis. PHP dapat menerima permintaan HTTP, memproses data yang dikirimkan client, menjalankan perintah terhadap basis data, serta mengembalikan respons kepada client.

Dalam pengembangan RESTful API, PHP digunakan untuk menangani metode HTTP, melakukan validasi input, mengelola data, dan menyusun respons dalam format JSON.

2.6 MySQL

MySQL merupakan sistem manajemen basis data relasional yang digunakan untuk menyimpan dan mengelola data dalam tabel. Data disimpan dalam baris dan kolom serta dapat dihubungkan dengan tabel lain sesuai rancangan basis data.

Dalam project Sistem Informasi Akademik, MySQL digunakan untuk menyimpan data mahasiswa beserta informasi lain yang diperlukan aplikasi. API bertindak sebagai perantara yang memproses permintaan client sebelum melakukan operasi terhadap basis data.

2.7 PHP Data Objects (PDO)

PHP Data Objects (PDO) merupakan antarmuka PHP yang digunakan untuk mengakses berbagai jenis basis data. PDO menyediakan mekanisme koneksi serta eksekusi perintah SQL melalui objek dan metode yang telah disediakan.

Penggunaan prepared statement membantu memisahkan struktur perintah SQL dari nilai input pengguna sehingga dapat mengurangi risiko SQL injection apabila diterapkan dengan benar. PDO juga mendukung penanganan kesalahan dalam proses akses basis data.

2.8 Model-View-Controller (MVC)

Model-View-Controller (MVC) merupakan pola arsitektur yang memisahkan tanggung jawab aplikasi menjadi tiga bagian, yaitu Model, View, dan Controller.

Model bertanggung jawab mengelola data serta berinteraksi dengan basis data. Controller menerima permintaan dan mengatur alur pemrosesan aplikasi. View bertugas menampilkan informasi kepada pengguna. Pada RESTful API yang mengembalikan JSON, respons dapat disusun oleh controller tanpa harus menampilkan halaman HTML.

Pemisahan tanggung jawab tersebut membantu meningkatkan keteraturan kode dan mempermudah pengembangan aplikasi.

2.9 Router dan Front Controller

Router merupakan komponen yang menentukan bagian aplikasi yang harus menangani permintaan berdasarkan URL dan metode HTTP. Router membantu mengarahkan permintaan menuju controller atau proses API yang sesuai.

Front controller merupakan titik masuk utama aplikasi yang menerima permintaan sebelum meneruskannya ke mekanisme pemrosesan berikutnya. Pada aplikasi PHP, file front controller biasanya ditempatkan di direktori `public`.

Kedua komponen ini membantu mengatur alur permintaan sehingga pengelolaan endpoint menjadi lebih terstruktur.

2.10 Postman

Postman merupakan aplikasi yang digunakan untuk menguji API dengan mengirimkan permintaan HTTP kepada server. Aplikasi ini mendukung metode GET, POST, PUT, dan DELETE, pengaturan header, pengiriman body JSON, serta pemeriksaan status HTTP dan respons server.

Postman memudahkan pengembang memeriksa fungsi API secara langsung sebelum API digunakan atau diintegrasikan dengan aplikasi client.

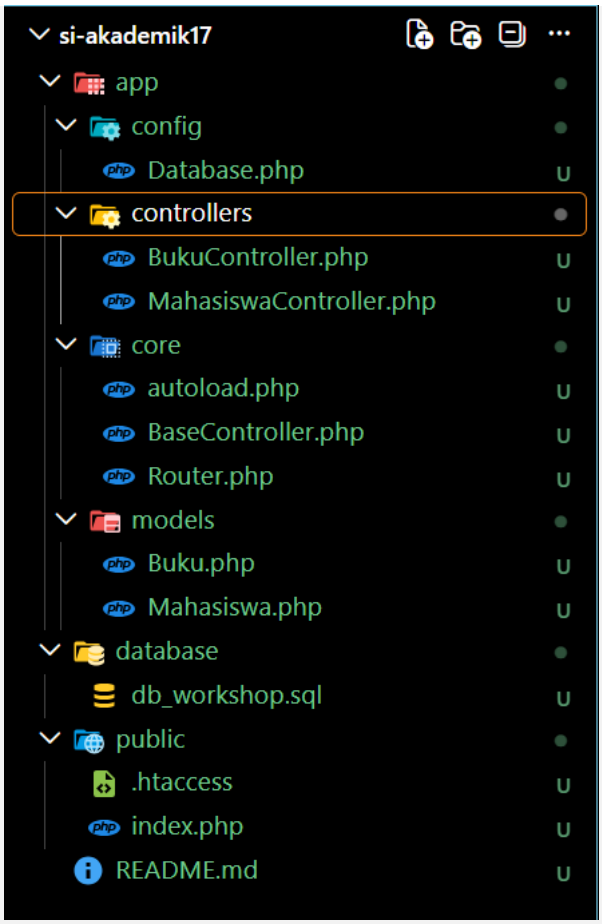
BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktikum

3.1.1 Studi Kasus

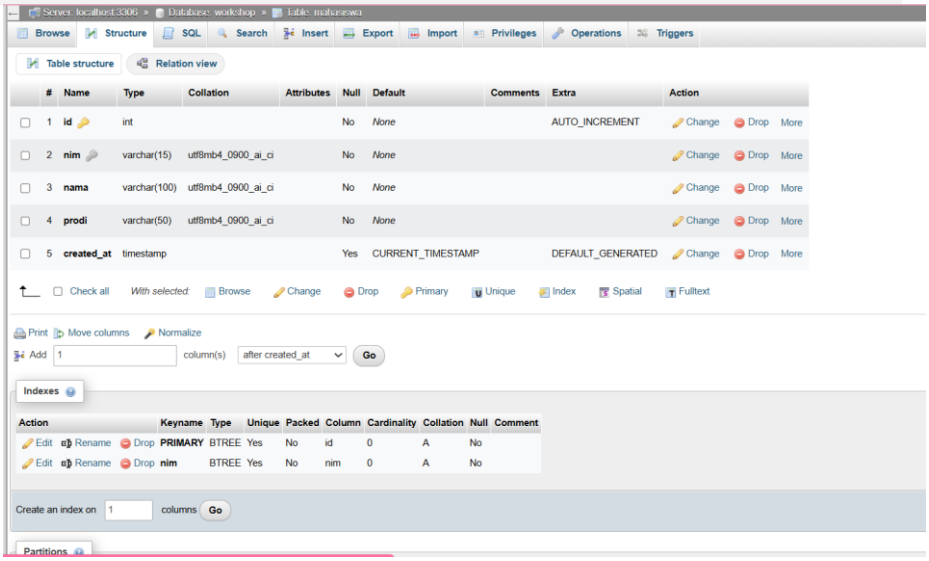
A. Struktur Folder Proyek Berpola MVC

Proyek bernama si-akademik17 diletakkan pada folder www. Folder app/ berisi kode aplikasi yang tidak diakses langsung oleh browser, sedangkan folder public/ adalah satu-satunya folder yang diakses browser.



B. Basis Data dan Tabel

Basis data yang digunakan dalam proyek ini adalah `workshop` dengan tabel `mahasiswa` dan `buku`. Tabel mahasiswa digunakan sebagai resource utama untuk mengelola data mahasiswa melalui RESTful API, sedangkan tabel buku digunakan untuk menerapkan pengembangan resource pada tugas mandiri. Tabel mahasiswa memiliki kolom `id`, `nim`, `nama`, `email`, `prodi`, dan `created_at`. Kolom `id` digunakan sebagai identitas data, sedangkan kolom `nim` memiliki batasan unik untuk mencegah penggunaan NIM yang sama. Pengelolaan data dilakukan melalui Model yang mengakses basis data menggunakan PDO dan prepared statement.



C. Kode Pemrograman Koneksi Database (Database.php)

Koneksi dibangun sebagai class dengan pola Singleton sederhana, sehingga satu koneksi PDO dipakai ulang oleh semua Model.

```php
<?php
class Database {
    private static ?PDO $connection = null;

    public static function getConnection(): PDO {
        if (self::$connection === null) {
            $host = 'localhost';
            $db   = 'workshop';
            $charset = 'utf8mb4';
```

```
            $dsn =
"mysql:host=$host;dbname=$db;charset=$charset";
            $options = [
                PDO::ATTR_ERRMODE            =>
PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE =>
PDO::FETCH_ASSOC,
            ];
            try {
                self::$connection = new PDO($dsn, 'root',
'', $options);
            } catch (PDOException $e) {
                http_response_code(500);
                header('Content-Type: application/json;
charset=utf-8');
                echo json_encode(['success' => false,
'message' => 'Koneksi database gagal']);
                exit;
            }
        }
        return self::$connection;
    }
}
```

D. Kode Pemrograman Model (Mahasiswa.php)

Controller bertugas menerima permintaan dari client, memanggil method pada Model, serta mengirimkan respons dalam format JSON dengan status HTTP yang sesuai. Method `index()` digunakan untuk mengambil seluruh data atau mencari data mahasiswa berdasarkan ID melalui metode GET. Method `store()` menangani penambahan data menggunakan POST, method `update()` menangani perubahan data menggunakan PUT, sedangkan method `destroy()` menangani penghapusan data menggunakan DELETE. Selain itu, Controller melakukan validasi terhadap field yang dikirimkan client, seperti NIM, nama, prodi, dan email. Jika NIM sudah terdaftar, Controller menangkap `DomainException` dari Model dan mengembalikan status HTTP 409 Conflict.

```
<?php
class Mahasiswa {
    private PDO $db;
```

```php
    public function __construct() {
        $this->db = Database::getConnection();
    }

    public function getAll(): array {
        return $this->db->query('SELECT * FROM mahasiswa
ORDER BY id DESC')->fetchAll();
    }

    public function find(int $id) {
        $stmt = $this->db->prepare('SELECT * FROM
mahasiswa WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->fetch();
    }

    /**
     * Validasi Tugas Mandiri 2: NIM tidak boleh
duplikat.
     * $exceptId dipakai saat update agar data itu
sendiri tidak dianggap duplikat.
     */
    public function nimExists(string $nim, ?int $exceptId
= null): bool {
        $sql = 'SELECT COUNT(*) FROM mahasiswa WHERE nim
= :nim';
        $params = [':nim' => $nim];
        if ($exceptId !== null) {
            $sql .= ' AND id <> :id';
            $params[':id'] = $exceptId;
        }
        $stmt = $this->db->prepare($sql);
        $stmt->execute($params);
        return (int) $stmt->fetchColumn() > 0;
    }
```

```
    public function create(array $data): string {
        if ($this->nimExists($data['nim'])) {
            throw new DomainException('NIM sudah
terdaftar');
        }
        $stmt = $this->db->prepare(
            'INSERT INTO mahasiswa (nim, nama, prodi)
VALUES (:nim, :nama, :prodi)'
        );
        $stmt->execute([':nim' => $data['nim'], ':nama'
=> $data['nama'], ':prodi' => $data['prodi']]);
        return $this->db->lastInsertId();
    }

    public function update(int $id, array $data): void {
        if ($this->nimExists($data['nim'], $id)) {
            throw new DomainException('NIM sudah
digunakan mahasiswa lain');
        }
        $stmt = $this->db->prepare(
            'UPDATE mahasiswa SET nim = :nim, nama
= :nama, prodi = :prodi WHERE id = :id'
        );
        $stmt->execute([':nim' => $data['nim'], ':nama'
=> $data['nama'], ':prodi' => $data['prodi'], ':id' =>
$id]);
    }

    public function delete(int $id): void {
        $stmt = $this->db->prepare('DELETE FROM mahasiswa
WHERE id = :id');
        $stmt->execute([':id' => $id]);
    }
}
```

E. Kode Pemrograman Controller (MahasiswaController.php)

Controller tidak menulis perintah SQL, hanya memanggil method Model lalu membentuk response. Method index() melayani GET, store() melayani POST, update() melayani PUT, dan destroy() melayani DELETE. Seluruh helper response (send, readJson, requireId, validationError) berada pada class induk BaseController.

```php
<?php
class MahasiswaController extends BaseController {
    private Mahasiswa $model;

    public function __construct() {
        $this->model = new Mahasiswa();
    }

    public function index($id = null): void {
        if ($id !== null && $id !== '') {
            $data =
$this->model->find($this->requireId($id));
            if (!$data) {
                $this->send(404, ['success' => false,
'message' => 'Data tidak ditemukan']);
            }
            $this->send(200, ['success' => true, 'data'
=> $data]);
        }

        $this->send(200, ['success' => true, 'data' =>
$this->model->getAll()]);
    }

    // POST /mahasiswa
    public function store(): void {
        $data = $this->clean($this->readJson());

        try {
            $id = $this->model->create($data);
```

```
        } catch (DomainException $e) {
            $this->send(409, ['success' => false,
'message' => $e->getMessage()]);
        }

        $this->send(201, [
            'success' => true,
            'message' => 'Data berhasil ditambahkan',
            'data'    => ['id' => (int) $id],
        ]);
    }

    // PUT /mahasiswa?id=1
    public function update($id = null): void {
        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            $this->send(404, ['success' => false,
'message' => 'Data tidak ditemukan']);
        }

        $data = $this->clean($this->readJson());

        try {
            $this->model->update($id, $data);
        } catch (DomainException $e) {
            $this->send(409, ['success' => false,
'message' => $e->getMessage()]);
        }

        $this->send(200, ['success' => true, 'message' =>
'Data berhasil diperbarui']);
    }

    // DELETE /mahasiswa?id=1
    public function destroy($id = null): void {
```

```
        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            $this->send(404, ['success' => false,
'message' => 'Data tidak ditemukan']);
        }

        $this->model->delete($id);
        $this->send(200, ['success' => true, 'message' =>
'Data berhasil dihapus']);
    }

    /** Validasi field wajib dan panjang maksimal sesuai
kolom tabel. */
    private function clean(array $input): array {
        $data = [
            'nim'   => $this->str($input, 'nim'),
            'nama'  => $this->str($input, 'nama'),
            'email' => $this->str($input, 'email'),
            'prodi' => $this->str($input, 'prodi'),
        ];

        if ($data['nim'] === '' || $data['nama'] === ''
|| $data['prodi'] === '') {
            $this->send(400, ['success' => false,
'message' => 'Field nim, nama, dan prodi wajib diisi']);
        }
        if ($this->len($data['nim']) > 15 ||
$this->len($data['nama']) > 100 ||
$this->len($data['prodi']) > 50) {
            $this->send(400, ['success' => false,
'message' => 'Panjang maksimal: nim 15, nama 100, prodi
50 karakter']);
        }
```

```
        if ($data['email'] !== '') {
            if (!filter_var($data['email'],
FILTER_VALIDATE_EMAIL) || $this->len($data['email']) >
100) {
                $this->send(400, ['success' => false,
'message' => 'Format email tidak valid']);
            }
        } else {
            $data['email'] = null;
        }

        return $data;
    }
}
```

F. Kode Pemrograman Autoloader dan Router

Autoloader memuat class secara otomatis dari folder config, core, models, dan controllers. Router membaca URL serta HTTP method, mencari nama resource pada peta, lalu memanggil method Controller yang sesuai.

1) `app/core/autoload.php`

```
<?php
spl_autoload_register(function ($class) {
    $folders = ['config', 'core', 'models',
'controllers'];
    foreach ($folders as $folder) {
        $path = __DIR__ .
"/../{$folder}/{$class}.php";
        if (file_exists($path)) {
            require $path;
            return;
        }
    }
});
```

2) app/core/Router.php

```php
<?php
class Router {

    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',
    ];

    public function dispatch(): void {
        try {
            $uri =
trim(parse_url($_SERVER['REQUEST_URI'],
PHP_URL_PATH), '/');
            $segments = $uri === '' ? [] :
explode('/', $uri);

            // Cari nama resource pada URL; segmen
sesudahnya (jika ada) dianggap ID
            $resource = null;
            $pathId = null;
            foreach ($segments as $i => $segment) {
                if (isset($this->map[$segment])) {
                    $resource = $segment;
                    $pathId = $segments[$i + 1] ??
null;
                    break;
                }
            }

            if ($resource === null) {
                $this->json(404, ['success' => false,
'message' => 'Endpoint tidak ditemukan']);
            }

            $id = $pathId ?? ($_GET['id'] ?? null);
```

```
            $controllerName = $this->map[$resource];
            $controller = new $controllerName();

            switch ($_SERVER['REQUEST_METHOD']) {
                case
'GET':    $controller->index($id); break;
                case 'POST':   $controller->store();
break;
                case 'PUT':
                case
'PATCH':  $controller->update($id); break;
                case 'DELETE':
$controller->destroy($id); break;
                default:
                    header('Allow: GET, POST, PUT,
PATCH, DELETE');
                    $this->json(405, ['success' =>
false, 'message' => 'Method tidak diizinkan']);
            }
        } catch (Throwable $e) {
            error_log($e->getMessage());
            $this->json(500, ['success' => false,
'message' => 'Terjadi kesalahan pada server']);
        }
    }

    private function json(int $status, array $body):
void {
        http_response_code($status);
        header('Content-Type: application/json;
charset=utf-8');
        echo json_encode($body, JSON_PRETTY_PRINT |
JSON_UNESCAPED_UNICODE);
        exit;
    }
}
```

G. Kode Pemrograman Front Controller (index.php dan .htaccess)

Berkas .htaccess mengarahkan semua request yang bukan berupa berkas atau folder statis ke index.php. Dengan demikian endpoint dapat diakses tanpa ekstensi .php.

1) `public/index.php`

```php
<?php
require_once __DIR__ . '/../app/core/autoload.php';

(new Router())->dispatch();
```

2) `public/.htaccess`

```
RewriteEngine On

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

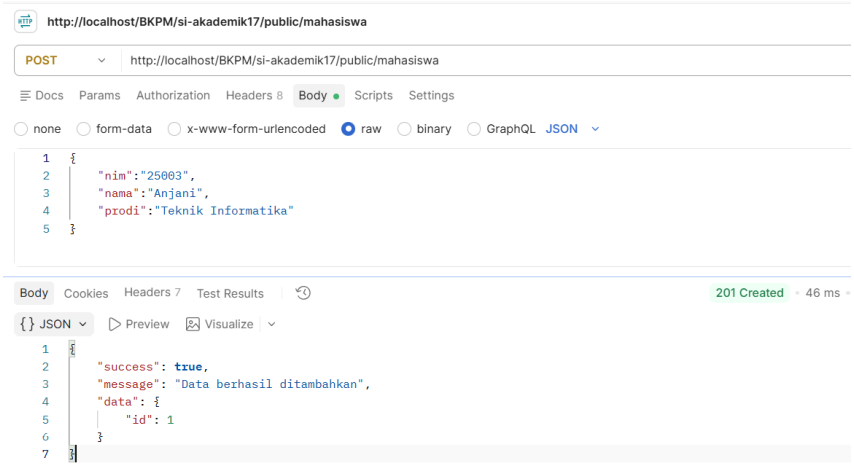
H. Pengujian API Menggunakan Postman

Pengujian dilakukan menggunakan Postman dengan alamat dasar http://localhost/BKPM/si-akademik17/public/. Setiap pengujian ditelusuri alur eksekusinya, misalnya pada POST: Router meneruskan ke MahasiswaController::store(), memanggil Mahasiswa::create() pada lapisan Model.

a) Create: menambah data (POST)

Item	Isi
Method	POST
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa
Body (JSON)	{ "nim":"25003", "nama":"Anjani", "prodi":"Teknik Informatika" }

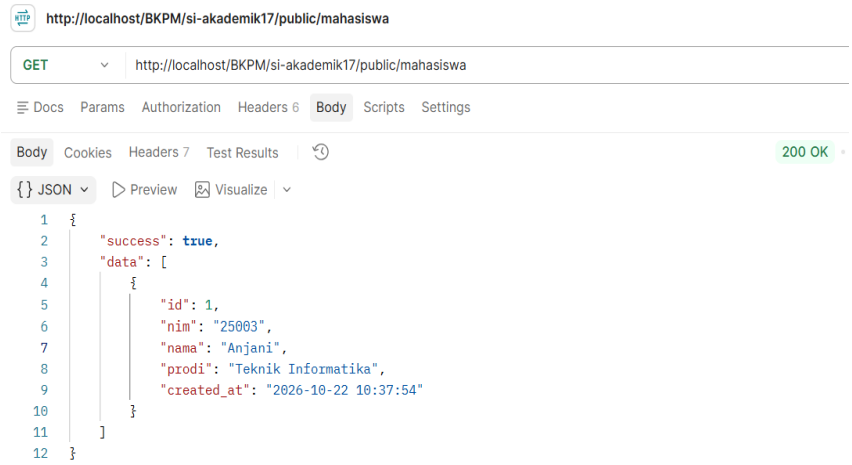
Hasil yang diharapkan: status 201 Created, success bernilai true, dan data berisi mahasiswa yang baru ditambahkan lengkap dengan id-nya.



b) Read: menampilkan seluruh data (GET)

Item	Isi
Method	GET
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa
Body (JSON)	-

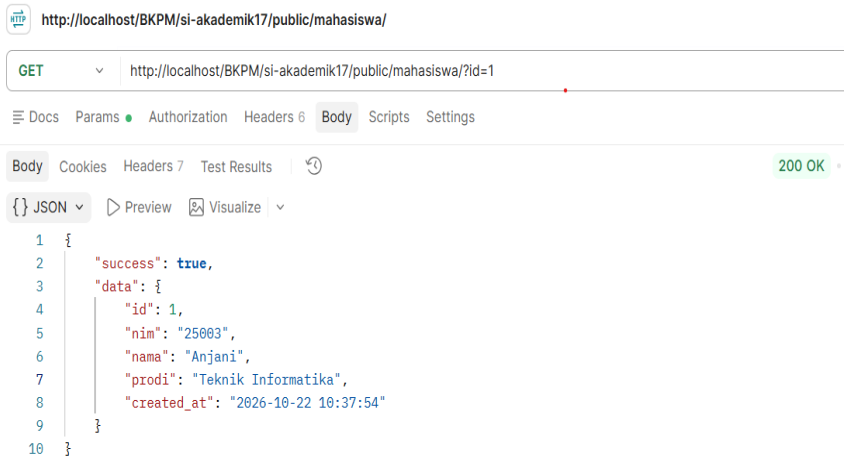
Hasil yang diharapkan: status 200 OK dan data berisi daftar seluruh mahasiswa.



c) Read: menampilkan satu data (GET ?id=)

Item	Isi
Method	GET
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa/?id=1
Body (JSON)	-

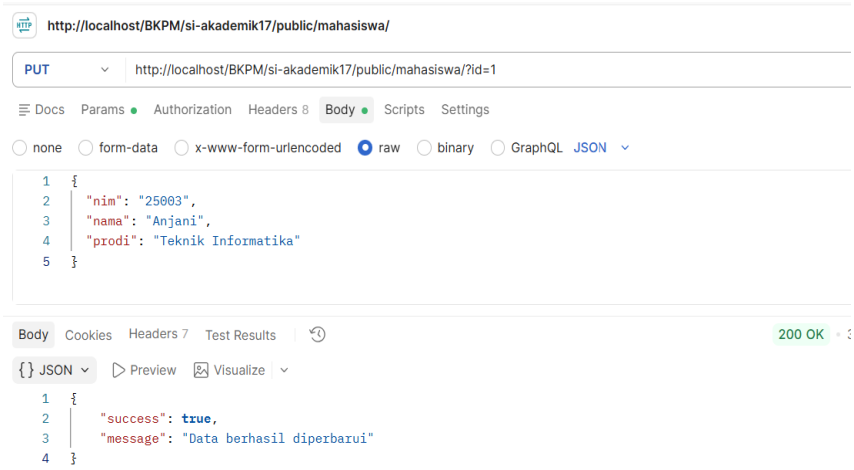
Hasil yang diharapkan: status 200 OK dan data berisi satu mahasiswa sesuai id.



d) Update: mengubah data (PUT)

Item	Isi
Method	PUT
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa/?id=1
Body (JSON)	{ "nim": "25003", "nama": "Anjani", "prodi": "Teknik Informatika" }

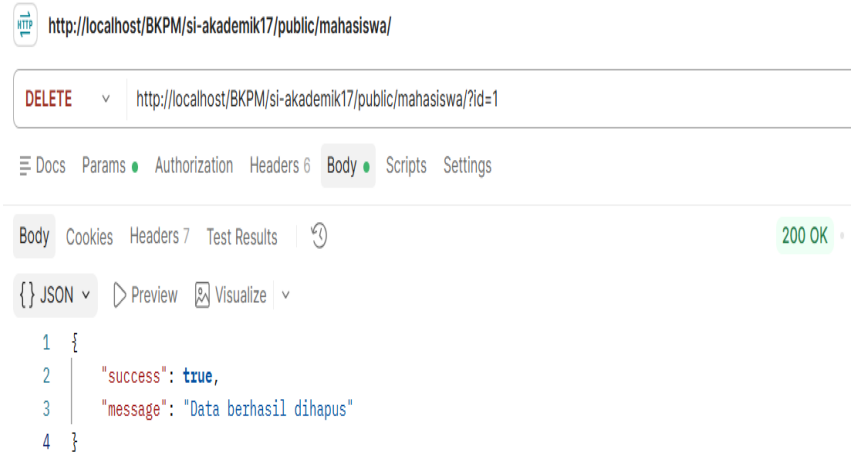
Hasil yang diharapkan: status 200 OK dengan pesan Data berhasil diperbarui.



e) Delete: menghapus data (DELETE)

Item	Isi
Method	DELETE
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa/?id=1
Body (JSON)	-

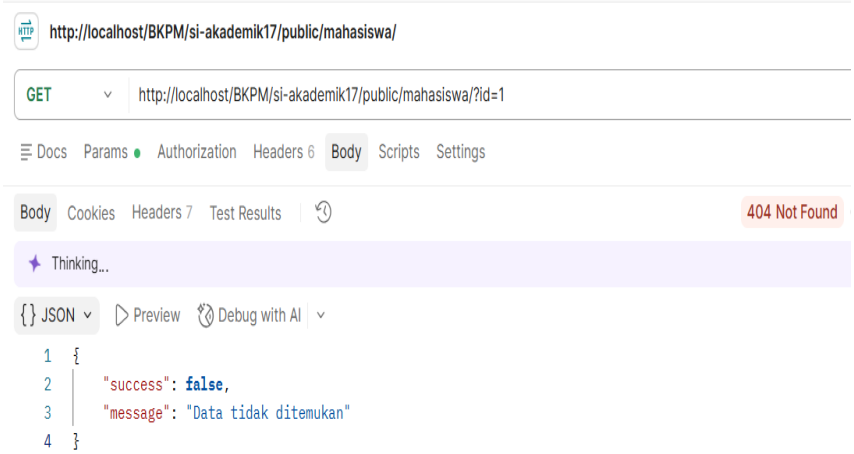
Hasil yang diharapkan: status 200 OK dengan pesan Data berhasil dihapus.



f) Memastikan data sudah terhapus

Item	Isi
Method	GET
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa/?id=1
Body (JSON)	-

Hasil yang diharapkan: status 404 Not Found dengan pesan Data tidak ditemukan.



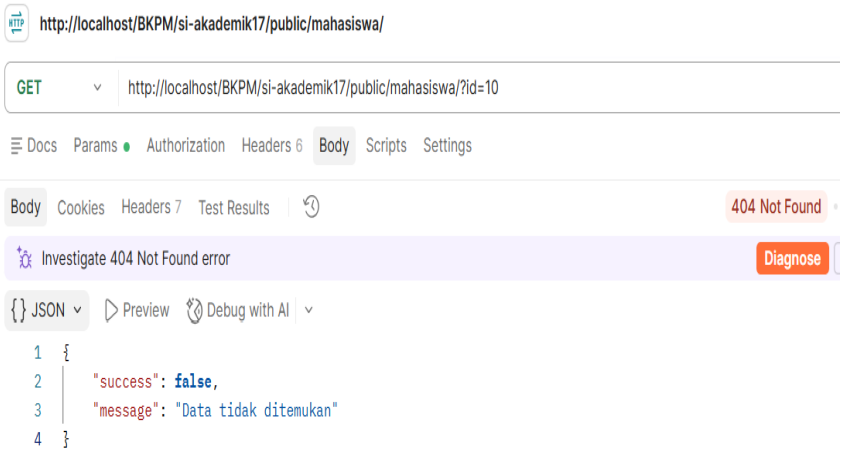
I. Pengujian Kondisi Error

Selain pengujian normal, dilakukan pengujian terhadap kondisi kesalahan untuk memastikan API dapat menangani permintaan yang tidak sesuai. Pengujian meliputi ID yang tidak ditemukan, NIM kosong, format email tidak valid, dan data JSON yang tidak lengkap. Respons kesalahan diberikan dalam format JSON disertai status HTTP yang sesuai, sehingga client dapat mengetahui penyebab permintaan gagal.

a) ID tidak ditemukan

Item	Isi
Method	GET
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa/?id=10
Body (JSON)	-

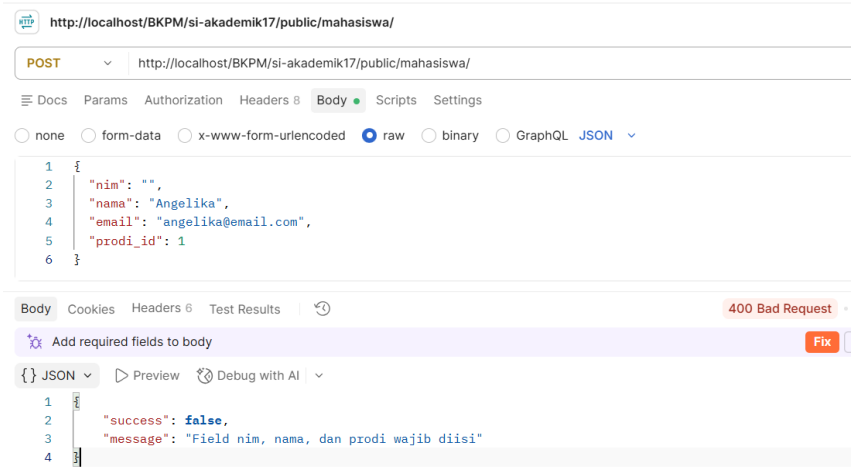
Hasil yang diharapkan: status 404 Not Found dengan pesan Data tidak ditemukan. Hasil tersebut berlaku jika ID 10 memang tidak tersedia di database.



b) NIM kosong

Item	Isi
Method	POST
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa/?id=10
Body (JSON)	{ "nim": "", "nama": "Angelika", "email": "angelika@email.com", "prodi_id": 1 }

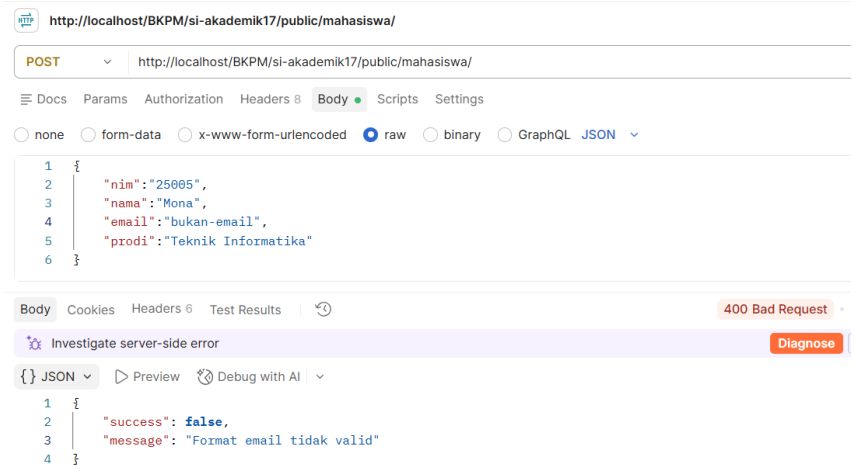
Hasil yang diharapkan: status 400 Bad Request, message Data tidak valid, dan errors.nim berisi NIM wajib diisi.



c) Email tidak valid

Item	Isi
Method	POST
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa/
Body (JSON)	{"nama":"Tes Error"}

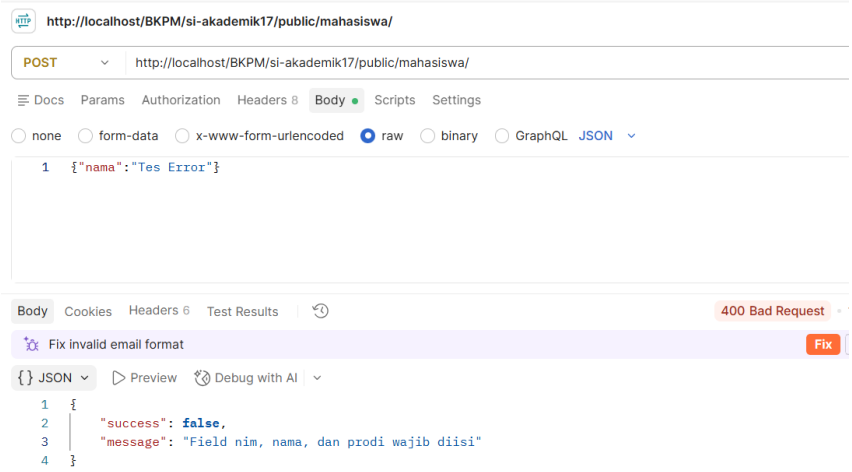
Hasil yang diharapkan: status 400 Bad Request dengan pesan Format email tidak valid, selama NIM yang digunakan tidak menimbulkan konflik. Jika NIM sudah terdaftar, gunakan NIM lain yang belum dipakai.



d) Data JSON tidak lengkap

Item	Isi
Method	POST
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa/
Body (JSON)	{ "nim":"25005", "nama":"Mona", "email":"bukan-email", "prodi":"Teknik Informatika" }

Hasil yang diharapkan: status 400 Bad Request dan errors berisi beberapa field yang wajib diisi (nim dan prodi).



3.2 Tugas Mandiri

3.2.1 Tugas Mandiri 1: Pengembangan Model dan Controller untuk Resource Buku

Pengembangan dilakukan dengan membuat model Buku.php dan controller BukuController.php menggunakan pola arsitektur MVC (Model-View-Controller). Model digunakan untuk mengelola data buku pada database, sedangkan controller menangani request dari client dan menghubungkannya dengan model. Resource buku kemudian didaftarkan pada peta Router agar dapat diakses melalui endpoint API.

A. Model Buku.php

Buat file `app/models/Buku.php`:

```php
<?php
class Buku {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }

    public function getAll(): array {
        return $this->db->query('SELECT * FROM buku
ORDER BY id DESC')->fetchAll();
    }

    public function find(int $id) {
        $stmt = $this->db->prepare('SELECT * FROM
buku WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->fetch();
    }

    public function create(array $data): string {
        $stmt = $this->db->prepare(
            'INSERT INTO buku (judul, penulis,
tahun_terbit) VALUES (:judul, :penulis, :tahun)'
        );
        $stmt->execute([
            ':judul'   => $data['judul'],
            ':penulis' => $data['penulis'],
            ':tahun'   => $data['tahun_terbit'],
        ]);
        return $this->db->lastInsertId();
    }

    public function update(int $id, array $data):
void {
        $stmt = $this->db->prepare(
```

```
            'UPDATE buku SET judul = :judul, penulis
= :penulis, tahun_terbit = :tahun WHERE id = :id'
        );
        $stmt->execute([
            ':judul'   => $data['judul'],
            ':penulis' => $data['penulis'],
            ':tahun'   => $data['tahun_terbit'],
            ':id'      => $id,
        ]);
    }

    public function delete(int $id): void {
        $stmt = $this->db->prepare('DELETE FROM buku
WHERE id = :id');
        $stmt->execute([':id' => $id]);
    }
}
```

Model tersebut menyediakan fungsi untuk mengambil seluruh data buku dan mencari buku berdasarkan ID. Kode ini mengasumsikan tabel buku memiliki kolom id. Pastikan nama tabel dan kolom sesuai dengan database yang digunakan.

B. Controller BukuController.php

Buat file `app/controllers/BukuController.php`:

```
<?php
class BukuController extends BaseController {
    private Buku $model;

    public function __construct() {
        $this->model = new Buku();
    }

    public function index($id = null): void {
        if ($id !== null && $id !== '') {
```

```
            $data =
$this->model->find($this->requireId($id));
            if (!$data) {
                $this->send(404, ['success' => false,
'message' => 'Data tidak ditemukan']);
            }
            $this->send(200, ['success' => true,
'data' => $data]);
        }

        $this->send(200, ['success' => true, 'data'
=> $this->model->getAll()]);
    }

    public function store(): void {
        $data = $this->clean($this->readJson());
        $id = $this->model->create($data);

        $this->send(201, [
            'success' => true,
            'message' => 'Data berhasil ditambahkan',
            'data'    => ['id' => (int) $id],
        ]);
    }

    public function update($id = null): void {
        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            $this->send(404, ['success' => false,
'message' => 'Data tidak ditemukan']);
        }

        $this->model->update($id,
$this->clean($this->readJson()));
```

```
        $this->send(200, ['success' => true,
'message' => 'Data berhasil diperbarui']);
    }

    public function destroy($id = null): void {
        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            $this->send(404, ['success' => false,
'message' => 'Data tidak ditemukan']);
        }

        $this->model->delete($id);
        $this->send(200, ['success' => true,
'message' => 'Data berhasil dihapus']);
    }

    private function clean(array $input): array {
        $judul   = $this->str($input, 'judul');
        $penulis = $this->str($input, 'penulis');
        $tahun   = $this->str($input,
'tahun_terbit');

        if ($judul === '' || $penulis === '') {
            $this->send(400, ['success' => false,
'message' => 'Field judul dan penulis wajib diisi']);
        }
        if ($this->len($judul) > 150 ||
$this->len($penulis) > 100) {
            $this->send(400, ['success' => false,
'message' => 'Panjang maksimal: judul 150, penulis
100 karakter']);
        }
        if ($tahun !== '' && (!ctype_digit($tahun) ||
(int) $tahun < 1000 || (int) $tahun > (int) date('Y')
+ 1)) {
```

```
            $this->send(400, ['success' => false,
'message' => 'tahun_terbit tidak valid']);
        }

        return ['judul' => $judul, 'penulis' =>
$penulis, 'tahun_terbit' => $tahun === '' ? null :
(int) $tahun];
    }
}
```

Controller tersebut menangani request GET untuk menampilkan seluruh buku atau mencari buku berdasarkan ID. Kode ini mengikuti pola controller yang digunakan pada resource mahasiswa.

C. Pendaftaran resource pada Router

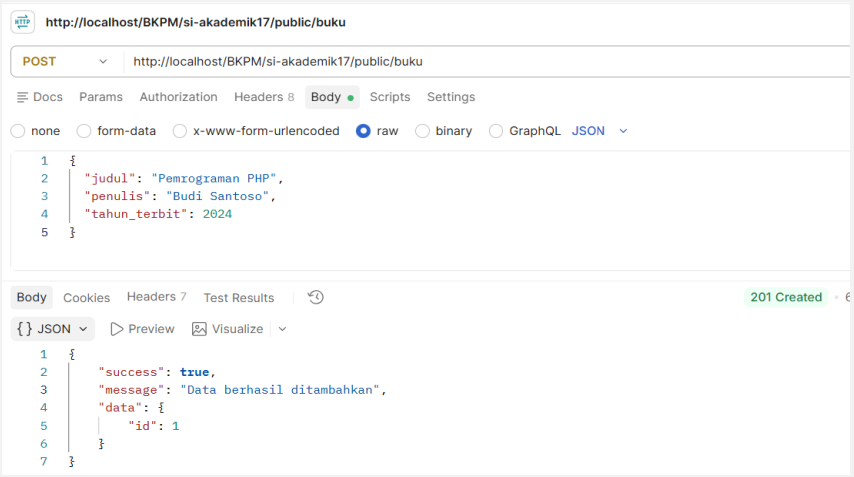
Buka file `app/core/Router.php`. Pastikan resource buku sudah terdaftar dalam properti $map:

```
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',
    ];
```

Router yang kamu kirim sebelumnya sudah memiliki pendaftaran resource buku, sehingga bagian ini tidak perlu ditambahkan lagi jika kodenya masih sama.

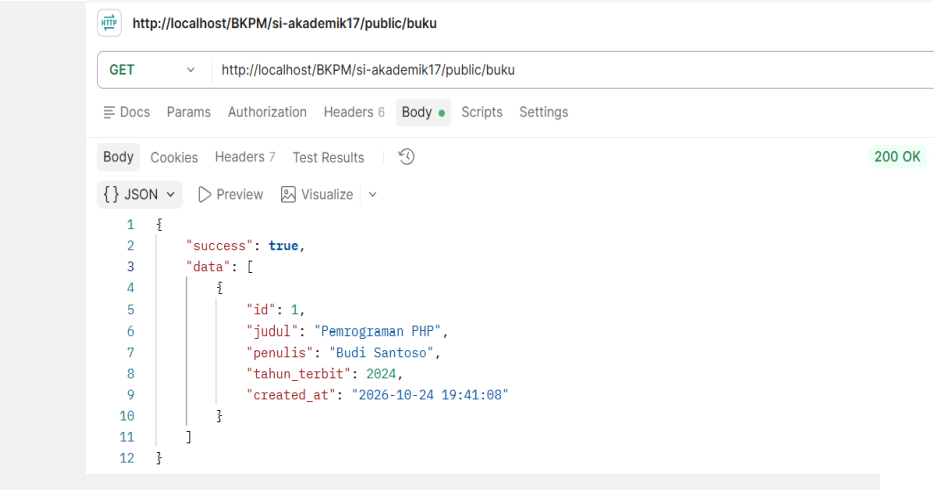
a) Create: menambah data (POST)

Item	Isi
Method	POST
Alamat	http://localhost/BKPM/si-akademik17/public/buku
Body (JSON)	{ "judul": "Pemrograman PHP", "penulis": "Budi Santoso", "tahun_terbit": 2024 }



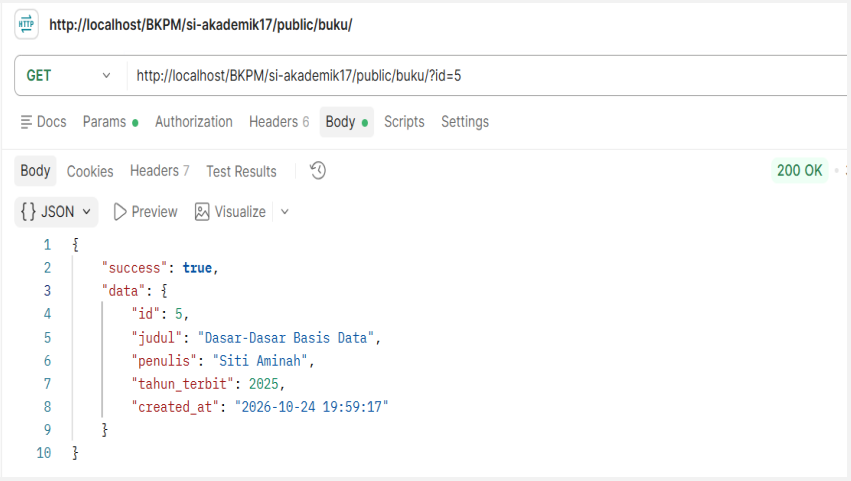
b) Read: menampilkan seluruh data (GET)

Item	Isi
Method	GET
Alamat	http://localhost/BKPM/si-akademik17/public/buku
Body (JSON)	-



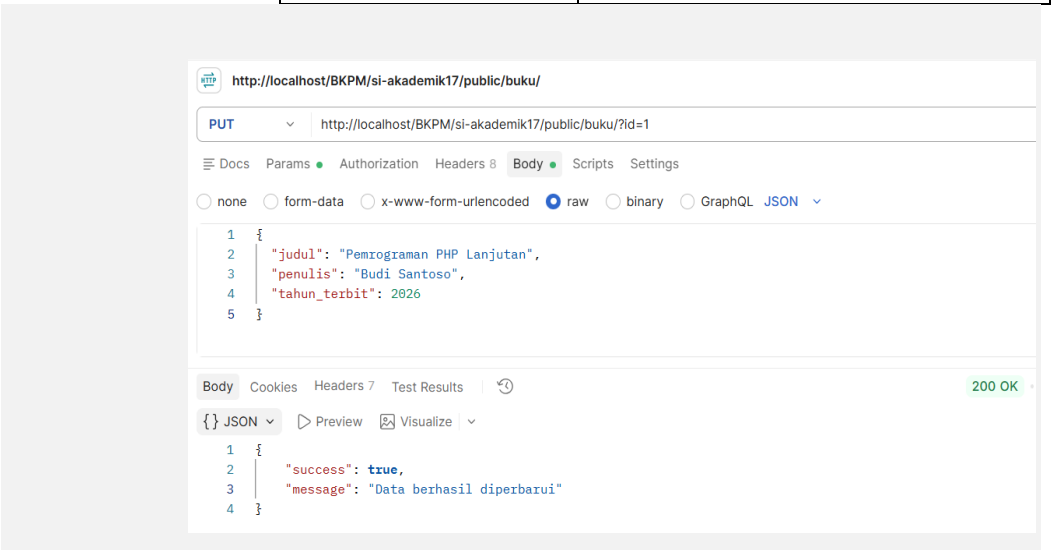
c) Read: menampilkan satu data (GET ?id=)

Item	Isi
Method	GET
Alamat	http://localhost/BKPM/si-akademik17/public/buku/?id=5
Body (JSON)	-



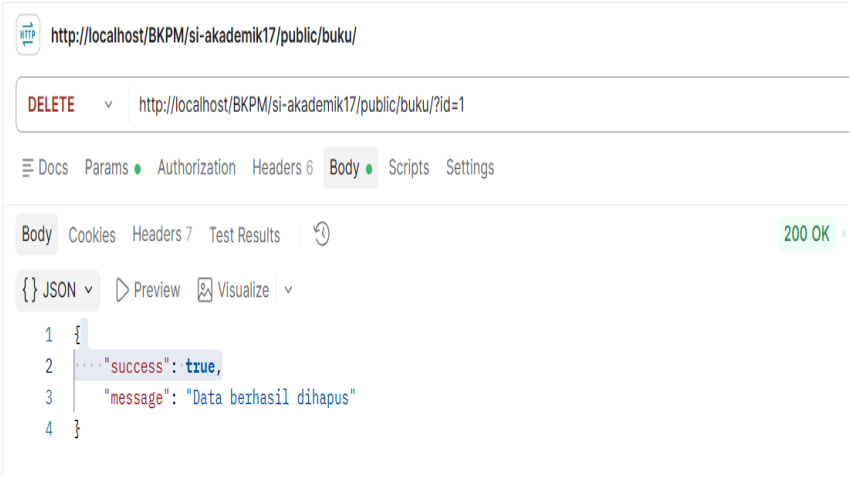
d) Update: mengubah data (PUT)

Item	Isi
Method	PUT
Alamat	http://localhost/BKPM/si-akademik17/public/buku/?id=1
Body (JSON)	{ "judul": "Pemrograman PHP Lanjutan", "penulis": "Budi Santoso", "tahun_terbit": 2026 }



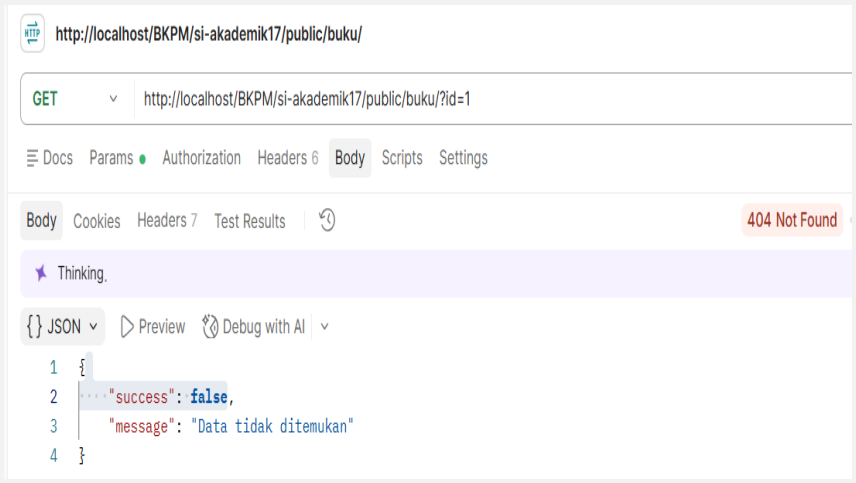
e) Delete: menghapus data (DELETE)

Item	Isi
Method	DELETE
Alamat	http://localhost/BKPM/si-akademik17/public/buku/?id=1
Body (JSON)	-



f) Memastikan data sudah terhapus

Item	Isi
Method	GET
Alamat	http://localhost/BKPM/si-akademik17/public/buku/?id=1
Body (JSON)	-



3.2.2 Tugas Mandiri 2: Validasi Duplikasi NIM pada Model Mahasiswa

Validasi dilakukan pada model Mahasiswa.php agar NIM yang sama tidak dapat didaftarkan lebih dari satu kali. Pada project kamu, fungsi nimExists() dan validasi di fungsi create() sebenarnya sudah tersedia.

a) Periksa kode `Mahasiswa.php`

Buka file app/models/Mahasiswa.php. Pastikan ada fungsi nimExists() berikut:

```
  public function nimExists(string $nim, ?int
$exceptId = null): bool {
        $sql = 'SELECT COUNT(*) FROM mahasiswa
WHERE nim = :nim';
        $params = [':nim' => $nim];
        if ($exceptId !== null) {
            $sql .= ' AND id <> :id';
            $params[':id'] = $exceptId;
        }
        $stmt = $this->db->prepare($sql);
        $stmt->execute($params);
        return (int) $stmt->fetchColumn() > 0;
    }
```

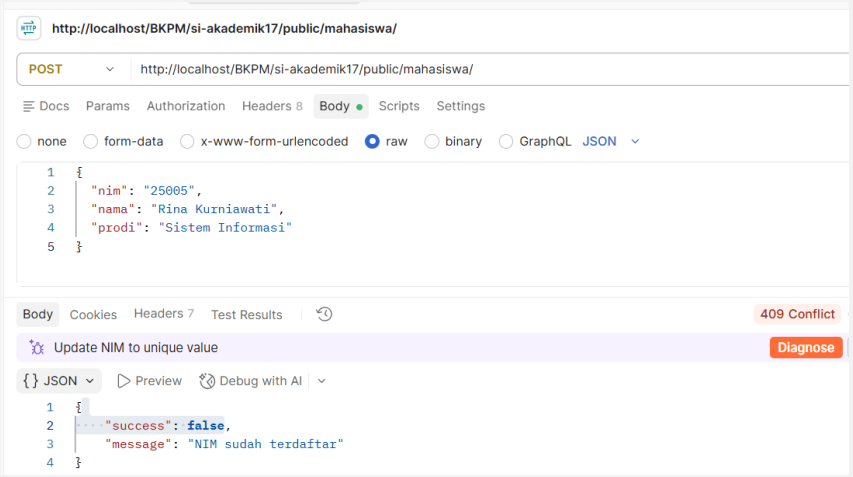
Kemudian, pastikan fungsi create() memeriksa NIM sebelum memasukkan data:

```
    public function create(array $data): string {
        if ($this->nimExists($data['nim'])) {
            throw new DomainException('NIM sudah
terdaftar');
        }
        $stmt = $this->db->prepare(
            'INSERT INTO mahasiswa (nim, nama,
prodi) VALUES (:nim, :nama, :prodi)'
        );
        $stmt->execute([':nim' => $data['nim'],
':nama' => $data['nama'], ':prodi' =>
$data['prodi']]);
        return $this->db->lastInsertId();
    }
```

b) Uji duplikasi NIM di Postman

Misalnya, NIM 25001 sudah terdaftar di tabel mahasiswa. Kirim request berikut:

Item	Isi
Method	POST
Alamat	http://localhost/BKPM/si-akademik17/public/mahasiswa
Body (JSON)	-



c) Kenapa statusnya 409?

Status HTTP 409 Conflict diberikan ketika permintaan tidak dapat diproses karena terjadi konflik dengan data yang sudah tersedia. Pada pengujian ini, Model memeriksa NIM menggunakan method `nimExists()`. Jika NIM sudah terdaftar, Model melemparkan `DomainException`, kemudian Controller menangkap kesalahan tersebut dan mengirimkan respons dengan status 409. Mekanisme ini mencegah penambahan data mahasiswa menggunakan NIM yang sama.

3.2.3 Tugas Mandiri 3: Dokumentasi Endpoint API dan Diagram Alur Router → Controller → Model.

A. Dokumentasi Endpoint API

Semua endpoint pada proyek ini menggunakan alamat dasar `http://localhost/BKPM/si-akademik17/public/`. Endpoint

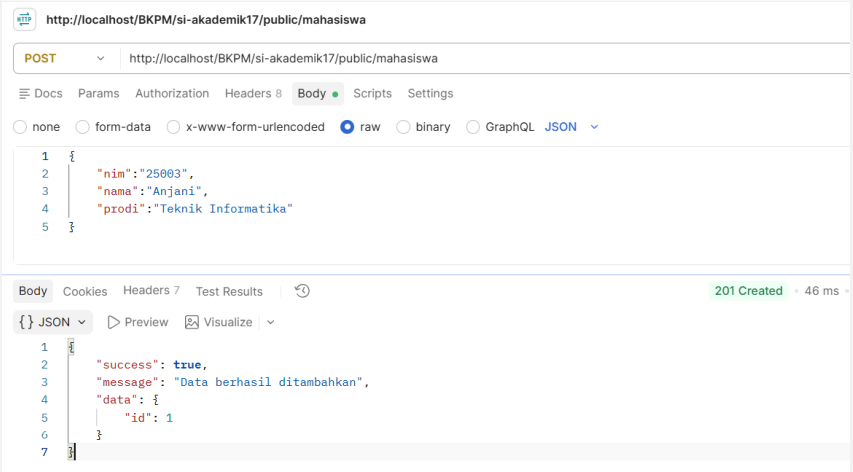
mahasiswa dan buku dapat diakses dengan menambahkan nama resource serta parameter ID apabila diperlukan.

Method Endpoint	Fungsi
GET: `/public/mahasiswa`	Menampilkan seluruh mahasiswa
GET: `/public/mahasiswa?id=1`	Menampilkan mahasiswa berdasarkan ID
POST: `/public/mahasiswa`	Menambahkan mahasiswa
PUT: `/public/mahasiswa?id=1`	Memperbarui data mahasiswa
DELETE: `/public/mahasiswa?id=1`	Menghapus data mahasiswa
GET: /public/buku	Menampilkan seluruh buku
GET: `/public/buku?id=1`	Menampilkan buku berdasarkan ID

Semua endpoint di atas menggunakan alamat dasar:

http://localhost/BKPM/si-akademik17/

Contoh request untuk menambahkan mahasiswa menggunakan metode POST:



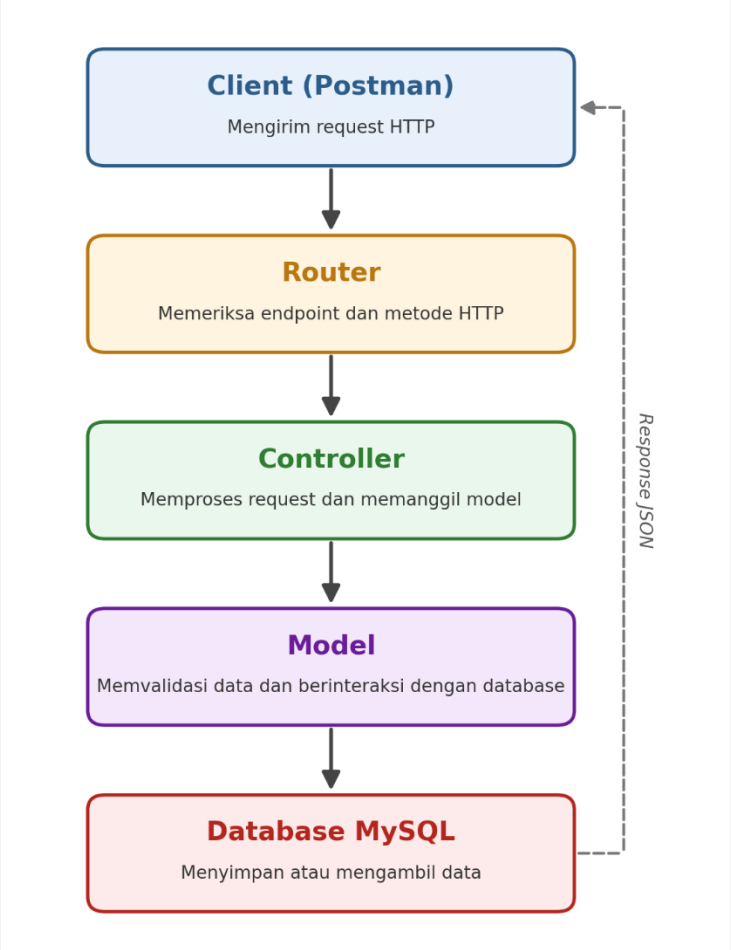
Respons berhasil di atas merupakan contoh dokumentasi. Nilai ID dan respons aktual bergantung pada hasil pengujian di database.

a) Tabel Kode Status HTTP pada Pengujian RESTful API

Kode	Arti	Contoh kasus
200	OK	GET, PUT, atau DELETE berhasil
201	Created	POST berhasil menambah data
400	Bad Request	NIM kosong, email tidak valid, JSON rusak atau kosong, ID tidak valid
404	Not Found	ID tidak ditemukan atau endpoint salah
405	Method Not Allowed	HTTP method yang tidak didukung
409	Conflict	NIM atau NIDN sudah dipakai
500	Internal Server Error	Kesalahan tak terduga pada server

B. Diagram Alur Router → Controller → Model

Diagram berikut menunjukkan alur pemrosesan request API berdasarkan pola MVC yang diterapkan.



Hasil pemrosesan dikembalikan melalui Controller dan Router kepada client dalam format JSON.

C. Kesimpulan Tugas Mandiri

Tugas mandiri ini menerapkan pengembangan resource baru menggunakan pola MVC, validasi duplikasi NIM pada model, serta dokumentasi endpoint API. Resource buku didaftarkan pada Router agar dapat diakses melalui endpoint yang sesuai. Dokumentasi dan diagram alur membantu menjelaskan proses komunikasi antara client, Router, Controller, Model, dan database.

BAB 4. PEMBAHASAN

4.1 Pembahasan

Praktikum Acara 17 membahas implementasi RESTful API pada Sistem Informasi Akademik menggunakan PHP dan MySQL sebagai lanjutan dari pengembangan API pada praktikum sebelumnya. Implementasi dilakukan dengan menerapkan metode HTTP sesuai fungsi masing-masing endpoint. Metode GET digunakan untuk mengambil data mahasiswa, POST untuk menambahkan data, PUT untuk memperbarui data, dan DELETE untuk menghapus data mahasiswa.

Penggunaan JSON sebagai format pertukaran data membuat informasi yang dikirimkan dan diterima oleh API tersusun secara terstruktur. Selain itu, penerapan router, controller, dan model membantu mengatur alur permintaan serta memisahkan proses pengolahan data dari penanganan permintaan. Dengan demikian, RESTful API dapat menjadi penghubung antara aplikasi client dan basis data melalui server.

Praktikum ini juga mencakup studi kasus pengelolaan data mahasiswa serta tugas praktik mandiri berupa pengembangan resource baru, validasi NIM agar tidak duplikat, dan dokumentasi endpoint API. Kegiatan tersebut bertujuan meningkatkan pemahaman mengenai pengembangan API yang terstruktur dan sesuai kebutuhan aplikasi.

4.2 Analisis Pengujian

Pengujian RESTful API dilakukan menggunakan Postman dengan mengirimkan permintaan melalui metode GET, POST, PUT, dan DELETE. Setiap pengujian bertujuan memastikan endpoint menjalankan fungsi yang sesuai serta menghasilkan respons JSON dan status kode HTTP yang tepat.

Pengujian GET digunakan untuk memastikan data mahasiswa dapat ditampilkan. Pengujian POST dilakukan untuk memeriksa penambahan data baru, sedangkan PUT digunakan untuk memeriksa perubahan data mahasiswa. Pengujian DELETE dilakukan untuk memastikan data dengan ID tertentu dapat dihapus. Selain pengujian normal, pengujian dapat dilakukan terhadap kondisi kesalahan,

seperti ID yang tidak ditemukan, format JSON yang tidak valid, dan NIM yang sudah terdaftar.

Hasil aktual dari setiap pengujian perlu dicocokkan dengan respons yang ditampilkan pada Postman serta kondisi data pada basis data. Dengan demikian, pengujian dapat digunakan untuk menilai kesesuaian fungsi API dengan kebutuhan pengelolaan data mahasiswa.

BAB 5. PENUTUP

5.1 Kesimpulan

Praktikum Acara 17 membahas implementasi RESTful API menggunakan PHP dan MySQL pada project Sistem Informasi Akademik. Materi yang diterapkan meliputi pengelolaan data mahasiswa melalui metode HTTP GET, POST, PUT, dan DELETE, pertukaran data menggunakan JSON, serta pengaturan alur aplikasi melalui router, controller, dan model. Pengujian menggunakan Postman membantu memeriksa permintaan dan respons API secara langsung. Melalui praktikum ini, mahasiswa dapat memahami proses komunikasi antara client dan server serta pentingnya validasi data dan penanganan kesalahan dalam pengembangan API.

5.2 Saran

Pengembangan RESTful API selanjutnya sebaiknya dilakukan dengan memperhatikan konsistensi struktur endpoint, validasi data, keamanan akses basis data, dan penanganan kesalahan. Setiap metode HTTP perlu diuji menggunakan berbagai kondisi, termasuk data yang tidak valid dan data yang tidak ditemukan. Dokumentasi endpoint juga perlu disusun secara lengkap agar API lebih mudah dipahami dan dikembangkan. Selain itu, pengujian ulang setelah perubahan kode perlu dilakukan untuk memastikan bahwa fungsi yang telah berjalan tetap berfungsi dengan baik.